

Project 5: Subqueries & Window Functions

20 queries (10 subqueries + 10 window functions) | Dataset: Northwind-style (7 tables) | 1 October 2026

Objective

Practice two more advanced SQL techniques on the same cleaned dataset: subqueries (a query nested inside another, used in WHERE, HAVING or FROM) and window functions (calculations across a set of rows that stay related to the current row, without collapsing the result like GROUP BY does). Ten queries cover each.

PART 1: SUBQUERIES

Query 1: Products priced above the overall average (scalar subquery)

```
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products)
ORDER BY Price DESC;
```

ProductName	Price
Grilled Chicken	30.00
Vanilla Extract	30.00
Jasmine Rice	25.00
Maple Syrup	25.00
Chef Anton's Cajun Seasoning	22.00
Clam Chowder	22.00
Chef Anton's Gumbo Mix	21.35
Vegetable Stir-Fry Mix	19.99
Chang	19.00
Chais	18.00
Chocolate Sauce	18.00
Granola Bars	16.00

12 row(s) returned.

Query 2: Products priced above their own category's average (correlated subquery)

```
SELECT p.ProductName, p.CategoryID, p.Price
FROM Products p
WHERE p.Price > (
    SELECT AVG(p2.Price) FROM Products p2 WHERE p2.CategoryID = p.CategoryID
)
ORDER BY p.CategoryID;
```

ProductName	CategoryID	Price
Chais	1	18.00
Chang	1	19.00
Vegetable Stir-Fry Mix	1	19.99
Clam Chowder	1	22.00
Chef Anton's Cajun Seasoning	2	22.00
Chef Anton's Gumbo Mix	2	21.35
Chocolate Sauce	2	18.00
Maple Syrup	2	25.00
Vanilla Extract	2	30.00
Grilled Chicken	4	30.00
Jasmine Rice	5	25.00
Almond Flour	5	11.50

12 row(s) returned.

Query 3: Most expensive product in each category (correlated subquery)

```
SELECT p.ProductName, p.CategoryID, p.Price
FROM Products p
WHERE p.Price = (
    SELECT MAX(p2.Price) FROM Products p2 WHERE p2.CategoryID = p.CategoryID
)
ORDER BY p.CategoryID;
```

ProductName	CategoryID	Price
Clam Chowder	1	22.00
Vanilla Extract	2	30.00
Granola Bars	3	16.00
Grilled Chicken	4	30.00
Jasmine Rice	5	25.00

5 row(s) returned.

Query 4: Categories whose average price beats the overall average (subquery in WHERE, on a derived table)

```
SELECT CategoryID, AVG(Price) AS avg_price
FROM Products
GROUP BY CategoryID
HAVING AVG(Price) > (SELECT AVG(Price) FROM Products)
ORDER BY avg_price DESC;
```

CategoryID	avg_price
2	16.77
1	16.36
4	16.35
3	16.00

4 row(s) returned.

Query 5: Suppliers who supply more products than the average supplier (subquery)

```
SELECT SupplierID, COUNT(*) AS product_count
FROM Products
GROUP BY SupplierID
HAVING COUNT(*) > (
    SELECT AVG(cnt) FROM (
        SELECT COUNT(*) AS cnt FROM Products GROUP BY SupplierID
    )
)
ORDER BY product_count DESC;
```

SupplierID	product_count
5	7
2	6
1	6

3 row(s) returned.

Query 6: Products that have never appeared in an order line (NOT IN subquery)

```
SELECT ProductID, ProductName
FROM Products
WHERE ProductID NOT IN (SELECT ProductID FROM OrderDetails)
ORDER BY ProductID;
```

ProductID	ProductName
1	Chais
2	Chang
3	Aniseed Syrup
4	Chef Anton's Cajun Seasoning
5	Chef Anton's Gumbo Mix
6	Apple Juice
7	Green Tea
8	Tomato Sauce
9	Grilled Chicken
15	Sesame Oil
16	Organic Oats
18	Peanut Butter

ProductID	ProductName
20	Clam Chowder
21	Mango Chutney
22	Maple Syrup

16 row(s) returned (showing first 15 of 16).

Query 7: Products that have appeared in at least one order line (IN subquery)

```
SELECT ProductID, ProductName
FROM Products
WHERE ProductID IN (SELECT ProductID FROM OrderDetails)
ORDER BY ProductID;
```

ProductID	ProductName
10	Instant Coffee
11	Chocolate Sauce
12	Whole Wheat Pasta
13	Cream Cheese
14	Basil Pesto
17	Vegetable Stir-Fry Mix
19	Jasmine Rice
24	Granola Bars
25	Vanilla Extract

9 row(s) returned.

Query 8: Customers who placed more orders than the average customer (subquery)

```
SELECT CustomerID, COUNT(*) AS order_count
FROM Orders
GROUP BY CustomerID
HAVING COUNT(*) > (
    SELECT AVG(cnt) FROM (
        SELECT COUNT(*) AS cnt FROM Orders GROUP BY CustomerID
    )
)
ORDER BY order_count DESC;
```

■ This returns 0 rows — and that's correct, not a bug. All 21 orders belong to 21 different customers, so no customer has more than 1 order; none can be “above average.” A query returning zero rows is still a valid result worth checking, not an error.

No rows matched — see note above.

0 row(s) returned.

Query 9: Second most expensive product (subquery with nested MAX)

```
SELECT ProductName, Price
FROM Products
WHERE Price = (
    SELECT MAX(Price) FROM Products
    WHERE Price < (SELECT MAX(Price) FROM Products)
);
```

ProductName	Price
Jasmine Rice	25.00
Maple Syrup	25.00

2 row(s) returned.

Query 10: Orders whose total line quantity beats the average order's total quantity (subquery in FROM)

```
SELECT OrderID, total_qty
FROM (
    SELECT OrderID, SUM(Quantity) AS total_qty FROM OrderDetails GROUP BY OrderID
) t
WHERE total_qty > (
    SELECT AVG(total_qty) FROM (
        SELECT SUM(Quantity) AS total_qty FROM OrderDetails GROUP BY OrderID
    )
)
ORDER BY total_qty DESC;
```

OrderID	total_qty
10249	49
10254	36
10250	28
10248	27
10251	27
10256	18

6 row(s) returned.

PART 2: WINDOW FUNCTIONS

Query 11: Rank all products by price, highest first (RANK)

```
SELECT ProductName, Price,
    RANK() OVER (ORDER BY Price DESC) AS price_rank
FROM Products
ORDER BY price_rank;
```

ProductName	Price	price_rank
Grilled Chicken	30.00	1
Vanilla Extract	30.00	1
Jasmine Rice	25.00	3
Maple Syrup	25.00	3
Chef Anton's Cajun Seasoning	22.00	5
Clam Chowder	22.00	5
Chef Anton's Gumbo Mix	21.35	7
Vegetable Stir-Fry Mix	19.99	8
Chang	19.00	9
Chais	18.00	10
Chocolate Sauce	18.00	10
Granola Bars	16.00	12
Tomato Sauce	15.00	13
Peanut Butter	15.00	13
Apple Juice	14.00	15

25 row(s) returned (showing first 15 of 25).

Query 12: Rank products within each category by price (ROW_NUMBER + PARTITION BY)

```
SELECT ProductName, CategoryID, Price,  
       ROW_NUMBER() OVER (PARTITION BY CategoryID ORDER BY Price DESC) AS rank_in_category  
FROM Products  
ORDER BY CategoryID, rank_in_category;
```

ProductName	CategoryID	Price	rank_in_category
Clam Chowder	1	22.00	1
Vegetable Stir-Fry Mix	1	19.99	2
Chang	1	19.00	3
Chais	1	18.00	4
Apple Juice	1	14.00	5
Green Tea	1	12.00	6
Instant Coffee	1	9.50	7
Vanilla Extract	2	30.00	1
Maple Syrup	2	25.00	2
Chef Anton's Cajun Seasoning	2	22.00	3
Chef Anton's Gumbo Mix	2	21.35	4
Chocolate Sauce	2	18.00	5
Tomato Sauce	2	15.00	6
Peanut Butter	2	15.00	7
Sesame Oil	2	11.00	8

25 row(s) returned (showing first 15 of 25).

Query 13: Dense-rank categories by how many products they contain (DENSE_RANK)

```
SELECT CategoryID, COUNT(*) AS product_count,  
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS count_rank  
FROM Products  
GROUP BY CategoryID  
ORDER BY count_rank;
```

CategoryID	product_count	count_rank
2	11	1
1	7	2
5	4	3
4	2	4
3	1	5

5 row(s) returned.

Query 14: Running total of quantity across all order lines (SUM OVER, ordered)

```
SELECT OrderDetailID, OrderID, Quantity,  
       SUM(Quantity) OVER (ORDER BY OrderDetailID) AS running_total  
FROM OrderDetails  
ORDER BY OrderDetailID;
```

OrderDetailID	OrderID	Quantity	running_total
1	10248	12	12
2	10248	10	22

OrderDetailID	OrderID	Quantity	running_total
3	10248	5	27
4	10249	9	36
5	10249	40	76
6	10250	8	84
7	10250	20	104
8	10251	15	119
9	10251	12	131
10	10252	10	141
11	10253	7	148
12	10253	9	157
13	10254	14	171
14	10254	22	193
15	10255	5	198

26 row(s) returned (showing first 15 of 26).

Query 15: Each product's price next to its category average (AVG OVER, PARTITION BY)

```
SELECT ProductName, CategoryID, Price,
       ROUND(AVG(Price) OVER (PARTITION BY CategoryID), 2) AS category_avg_price
FROM Products
ORDER BY CategoryID, Price DESC;
```

ProductName	CategoryID	Price	category_avg_price
Clam Chowder	1	22.00	16.36
Vegetable Stir-Fry Mix	1	19.99	16.36
Chang	1	19.00	16.36
Chais	1	18.00	16.36
Apple Juice	1	14.00	16.36
Green Tea	1	12.00	16.36
Instant Coffee	1	9.50	16.36
Vanilla Extract	2	30.00	16.77
Maple Syrup	2	25.00	16.77
Chef Anton's Cajun Seasoning	2	22.00	16.77
Chef Anton's Gumbo Mix	2	21.35	16.77
Chocolate Sauce	2	18.00	16.77
Tomato Sauce	2	15.00	16.77
Peanut Butter	2	15.00	16.77
Sesame Oil	2	11.00	16.77

25 row(s) returned (showing first 15 of 25).

Query 16: Each product's price compared to the next cheaper product (LAG)

```
SELECT ProductName, Price,
       LAG(Price) OVER (ORDER BY Price DESC) AS previous_price,
       Price - LAG(Price) OVER (ORDER BY Price DESC) AS price_gap
FROM Products
ORDER BY Price DESC;
```

ProductName	Price	previous_price	price_gap
Grilled Chicken	30.00	NULL	NULL
Vanilla Extract	30.00	30.00	0.00
Jasmine Rice	25.00	30.00	-5.00
Maple Syrup	25.00	25.00	0.00
Chef Anton's Cajun Seasoning	22.00	25.00	-3.00
Clam Chowder	22.00	22.00	0.00
Chef Anton's Gumbo Mix	21.35	22.00	-0.65
Vegetable Stir-Fry Mix	19.99	21.35	-1.36
Chang	19.00	19.99	-0.99
Chais	18.00	19.00	-1.00
Chocolate Sauce	18.00	18.00	0.00
Granola Bars	16.00	18.00	-2.00
Tomato Sauce	15.00	16.00	-1.00
Peanut Butter	15.00	15.00	0.00
Apple Juice	14.00	15.00	-1.00

25 row(s) returned (showing first 15 of 25).

Query 17: Each product's price compared to the next pricier product (LEAD)

```
SELECT ProductName, Price,
       LEAD(Price) OVER (ORDER BY Price DESC) AS next_price
FROM Products
ORDER BY Price DESC;
```

ProductName	Price	next_price
Grilled Chicken	30.00	30.00
Vanilla Extract	30.00	25.00
Jasmine Rice	25.00	25.00
Maple Syrup	25.00	22.00
Chef Anton's Cajun Seasoning	22.00	22.00
Clam Chowder	22.00	21.35
Chef Anton's Gumbo Mix	21.35	19.99
Vegetable Stir-Fry Mix	19.99	19.00
Chang	19.00	18.00
Chais	18.00	18.00
Chocolate Sauce	18.00	16.00
Granola Bars	16.00	15.00
Tomato Sauce	15.00	15.00
Peanut Butter	15.00	14.00
Apple Juice	14.00	12.00

25 row(s) returned (showing first 15 of 25).

Query 18: Split products into 4 price quartiles (NTILE)

```
SELECT ProductName, Price,
       NTILE(4) OVER (ORDER BY Price) AS price_quartile
FROM Products
ORDER BY price_quartile, Price;
```

ProductName	Price	price_quartile
Cream Cheese	2.70	1
Whole Wheat Pasta	3.50	1
Organic Oats	4.50	1
Basil Pesto	7.20	1
Instant Coffee	9.50	1
Mango Chutney	9.95	1
Aniseed Syrup	10.00	1
Sesame Oil	11.00	2
Almond Flour	11.50	2
Green Tea	12.00	2
Apple Juice	14.00	2
Tomato Sauce	15.00	2
Peanut Butter	15.00	2
Granola Bars	16.00	3
Chais	18.00	3

25 row(s) returned (showing first 15 of 25).

Query 19: Total quantity per order shown on every line, with each line's share (SUM OVER PARTITION BY)

```
SELECT OrderID, OrderDetailID, Quantity,
       SUM(Quantity) OVER (PARTITION BY OrderID) AS order_total_qty,
       ROUND(100.0 * Quantity / SUM(Quantity) OVER (PARTITION BY OrderID), 1) AS pct_of_order
FROM OrderDetails
ORDER BY OrderID, OrderDetailID;
```

OrderID	OrderDetailID	Quantity	order_total_qty	pct_of_order
10248	1	12	27	44.40
10248	2	10	27	37.00
10248	3	5	27	18.50
10249	4	9	49	18.40
10249	5	40	49	81.60
10250	6	8	28	28.60
10250	7	20	28	71.40
10251	8	15	27	55.60
10251	9	12	27	44.40
10252	10	10	10	100.00
10253	11	7	16	43.80
10253	12	9	16	56.30
10254	13	14	36	38.90

OrderID	OrderDetailID	Quantity	order_total_qty	pct_of_order
10254	14	22	36	61.10
10255	15	5	5	100.00

26 row(s) returned (showing first 15 of 26).

Query 20: Cheapest and priciest product name shown on every row per category (FIRST_VALUE, LAST_VALUE)

```
SELECT ProductName, CategoryID, Price,
       FIRST_VALUE(ProductName) OVER (PARTITION BY CategoryID ORDER BY Price ASC) AS cheapest_in_category,
       FIRST_VALUE(ProductName) OVER (PARTITION BY CategoryID ORDER BY Price DESC) AS priciest_in_category
FROM Products
ORDER BY CategoryID, Price;
```

ProductName	CategoryID	Price	cheapest_in_category	priciest_in_category
Instant Coffee	1	9.50	Instant Coffee	Clam Chowder
Green Tea	1	12.00	Instant Coffee	Clam Chowder
Apple Juice	1	14.00	Instant Coffee	Clam Chowder
Chais	1	18.00	Instant Coffee	Clam Chowder
Chang	1	19.00	Instant Coffee	Clam Chowder
Vegetable Stir-Fry Mix	1	19.99	Instant Coffee	Clam Chowder
Clam Chowder	1	22.00	Instant Coffee	Clam Chowder
Basil Pesto	2	7.20	Basil Pesto	Vanilla Extract
Mango Chutney	2	9.95	Basil Pesto	Vanilla Extract
Aniseed Syrup	2	10.00	Basil Pesto	Vanilla Extract
Sesame Oil	2	11.00	Basil Pesto	Vanilla Extract
Tomato Sauce	2	15.00	Basil Pesto	Vanilla Extract
Peanut Butter	2	15.00	Basil Pesto	Vanilla Extract
Chocolate Sauce	2	18.00	Basil Pesto	Vanilla Extract
Chef Anton's Gumbo Mix	2	21.35	Basil Pesto	Vanilla Extract

25 row(s) returned (showing first 15 of 25).

What I noticed

Subqueries

- A *correlated* subquery (queries 2, 3) re-runs once per outer row, referencing a column from the outer query — unlike a plain subquery, which runs once.
- A subquery in the FROM clause (query 10) is a derived table — it has to be aliased, and can itself be grouped, filtered or joined like a real table.
- NOT IN with a subquery (query 6) silently returns zero rows if the subquery can produce a NULL — worth checking the subquery column has no NULLs before relying on NOT IN.

Window functions

- Unlike GROUP BY, a window function keeps every row in the result — PARTITION BY groups the calculation, but doesn't collapse the rows.
- RANK() leaves gaps after ties (1, 2, 2, 4); DENSE_RANK() doesn't (1, 2, 2, 3); ROW_NUMBER() never ties, even when values do.

- LAG/LEAD look at a neighboring row without a self-join, which is the old way this was done before window functions existed.
- FIRST_VALUE needs its own ORDER BY inside the window — it does not reuse the outer query's ORDER BY.

Interview questions

What is a subquery? A query nested inside another query, used to supply a value, a list of values, or a derived table that the outer query filters, joins against, or selects from.

Subquery vs JOIN — when to use which? A JOIN combines columns from two tables into one row and is usually faster and clearer when you need data from both tables in the output. A subquery is a better fit when you only need to filter or compare against a value or set from another table, without pulling its columns into the result.

What is a window function? A calculation across a set of related rows (a “window”, defined with OVER()) that returns a value for every row, instead of collapsing rows into one summary row the way GROUP BY does.

RANK vs DENSE_RANK vs ROW_NUMBER? RANK skips numbers after a tie, DENSE_RANK doesn't, and ROW_NUMBER assigns a unique number to every row regardless of ties.